

UNIVERSITY OF BONN
CAISA LAB

NATURAL LANGUAGE PROCESSING LAB

SUMMER SEMESTER 2026

PROJECT PROPOSAL OF TEAM #

TEAM MEMBERS:

JIADONG HAN

3198936

MOHAMMAD SAEED MOTEVALI AMIN

50246276

June 1, 2026

1. Research Topic

Evaluation of multi-agent AI systems for scientific reasoning, with a focus on trajectory-level analysis of reasoning processes rather than only their final outputs. The project sits at the intersection of agentic AI evaluation, multi-agent system analysis, and applied scientific computing, with formal mathematics (LEAN 4) and astrophysical inference as testbeds.

2. Motivation and Objectives

2.1 Motivation

Agentic LLM systems are increasingly deployed in math and scientific workflows [1, 2, 3, 4], yet aggregate task success is a misleading proxy for reliability: agents pass statistical fit criteria while violating physical constraints [5], produce valid code that yields physically inconsistent results [6], and exhibit multi-step hallucinations qualitatively different from single-step LLM failures [7, 8].

Output-centric evaluation cannot say **where** an error originates, or **how** it propagates—particularly in *multi-agent* systems where coordination collapse, cross-agent propagation, and critic-masking are invisible to single-output metrics. In addition, it cannot deal with silent failures.

Motivated by [6], *silent failures* is defined operationally as a trial whose final output passes the output-level criterion but violates at least one trajectory-level anchor check: an astrophysical fit passing the statistical gate (ΔBIC , RMS) while failing the physical gate (Hungarian-matched parameter score, planet count) of [5].

Answering these requires *trajectory-level* evaluation, as motivated by [8]: we instrument the inter-agent communication graph and score both process and output. Since trajectory signals are not diagnostic by default—for example, raw retries may indicate useful exploration rather than failure—we treat structural metrics like *retry-success rate* as hypotheses to test.

STARGAZER [5] provides the closest empirical context. It shows that single-agent systems on radial-velocity tasks achieve good statistical fits but fail physical recovery, identifying alias convergence, perseveration, and format fragility through manual log inspection; we extend this with a multi-agent decomposition that they never test and with automatic detectors for those patterns, reusing their published single-agent results as our primary baseline. And in math, LEAN 4 [9] offers a stricter comparison regime where every reason is mechanically checkable. Studying both testbeds with comparable infrastructure makes *verification strictness* as the independent variable.

Research question: *How does the verifiability of intermediate reasoning steps shape multi-agent failure spectra, and can trajectory signals localise and detect failures that output evaluation misses?* Concretely: **(RQi)** does multi-agent decomposition relocate or transform the failures documented for single agents in [5], reducing perseveration through critic intervention, or introducing silent acceptance? **(RQii)** do trajectory signals (anchor violation rate, retry-success rate, cycle count) predict output failure *before* it manifests as stress increases? **(RQiii)** do structural failure signatures transfer across the two verification regimes, even where anchor logic differs

2.2 Objectives

We propose a **trajectory-level evaluation framework** for multi-agent scientific reasoning workflows which generalize STARGAZER [5]: it instruments inter-agent communication as a directed

event graph, scores both the reasoning process and the final artefact, and attributes anchor violations to specific events and agents. The trace schema and event logging are *framework-agnostic*; anchor logic, detectors, and evaluation criteria are *domain-adaptable*. Integrated with LEAN 4, the framework analyses mathematical reasoning failures such as proofs relying on **sorry**, skipped subgoals, or degenerate statements based on its stepwise verifiability [9]. Under our infrastructure, Lean interacts with the same way STARGAZER does, meaning automatic feedback, recomputable intermediate quantities, and systematically detectable failures. This yields three goals:

O1. Localisation infrastructure (primary): Deliver a non-invasive trace observer emitting a schema-validated event log and directed interaction graph, with a dashboard exposing trajectory views and anchor inspection points, so that any anchor violation is attributed to a specific event and originating agent. **O2.** Failure taxonomy and detectors (secondary). Define an operational taxonomy of at least five trajectory-level failure modes generalising the patterns of [5] (alias convergence, perseveration, format fragility) together with multi-agent-specific modes (coordination collapse, cross-agent error propagation, silent overconfidence / critic-masking), each with an automatic detector over the interaction graph and anchor labels. **O3.** Early prediction across verification regimes (stretch). Test whether trajectory signals degrade before output failure manifests, using the experimental design later.

3. Dataset

The two testbeds provide controlled tasks at contrasting levels of intermediate-step checkability. We use the following open-source datasets, which are publicly available.

Astrophysical inference (partially step-verifiable): Astrophysical inference tasks in the spirit of [5, 6] is used as the primary scientific benchmark for CMBAgent failure-study.

The dataset from [5] contains 100 synthetic systems and 20 real archival systems. They are distributed across difficulty scores from 1 to 10 with 12 tasks at each difficulty level. Grouped into tiers, this gives 36 easy tasks for scores 1-3, 48 medium tasks for scores 4-7, and 36 hard tasks for scores 8-10. [6] contains 14 CAMB One-Shot tasks and 5 deep research tasks.

Lean 4 theorem proving (step-verifiable): FATE [10] and LEANCAT [11] provide difficulty-tiered Lean benchmarks. FATE contains 350 formal algebra problems: 150 from undergraduate textbook, 100 graduate exam questions, and 100 PhD-level across broad algebra areas. LeanCat contains 100 Lean 4 category-theory problems with 20 Easy, 40 Medium, and 40 High.

4. Methodology

We will generalise STARGAZER’s [5] approach in the following ways, and design experiments across mathematics and physics. Our framework consists of framework-agnostic schema and logging layer, and domain-adaptable anchor logic, detectors, and evaluation criteria. At termination, each agent reports a numeric confidence, consumed by the overconfidence detector below.

1. Substrate and roles. We build on AG2 [12], with CMBAGENT [2] and a minimal Lean-agent [13] as reference configurations. A fixed four-role decomposition—planner, engineer/formaliser, critic/reviewer, executor/repairer—maps onto the standard scientific workflow: *hypothesise, implement, verify, run* loop.

2. Trace observer and interaction graph based on open source frameworks [14, 15]. The non-invasive observer of O1 hooks into the message-passing layer and records every message, tool call,

code event, retry, and execution result without modifying agent behaviour. Logs will become a directed graph with events as nodes and causal dependencies as edges. The schema and logging layer are framework-agnostic.

3. Anchor-based step evaluation. For **Lean**, we have a per-node correctness label with trajectory signals (definitions, tactic choices, abandoned subgoals) read from the proof state. For **astrophysics**, key intermediate quantities (periodogram peaks, residual RMS, per-component fits, posterior shapes) are re-computed from the STARGAZER forward model and compared to the agent’s reported values. The trajectory is scored by walking G in causal order and recording the first event whose anchor is violated, yielding error localisation (O1). *Success criterion of O1*: on both testbeds, the first violated anchor in each failed trial is correctly attributed to its originating event with precision and recall ≥ 0.8 against automatic ground truth (Lean kernel, STARGAZER forward model).

4. Failure detectors. The taxonomy of O2 is operationalised over G and the anchor labels: reasoning loops via cycle detection; coordination collapse via cross-agent reference density; silent overconfidence via the gap between elicited confidence and anchor-derived correctness; perseveration via *retry-success rate* (repeated retries producing the same wrong answer are pathology). Structural graph metrics are *exploratory*: their correlation with output failure is tested in O3, not assumed. The framework attributes failures to events and agents; it does not claim to explain the model-level cause. *Success criterion of O2*: detectors reproduce STARGAZER’s three patterns with precision and recall ≥ 0.8 against forward-model-derived labels.

5. Experimental design. Three axes: (i) *architecture*—single-agent REACT baseline vs. the four-role multi-agent configuration, isolating inter-agent effects and giving direct comparability to STARGAZER’s published single-agent results; (ii) *backbone*—small vs. large LLM, with *grounding* (Mathlib for Lean, CAMB docs for astrophysics) toggled within each; (iii) *stress conditions*—degenerate parameter regimes, weak priors, or missing intermediate information, the primary test of O3’s early-warning claim. Configurations run for $N \geq 10$ trials per task. *Success criterion of O3*: at least one trajectory signal shows significant degradation ($p < 0.05$, paired bootstrap with Bonferroni correction) one stress level before output pass rates drop.

5. Expected Results

We state four falsifiable predictions, each tied to an objective or sub-question of Section 2..

1. Critic intervention is mode-selective (RQ i, O2): The critic/reviewer agent reduces perseveration relative to the single-agent REACT baseline by interrupting repetitive behaviour, but does *not* reduce alias convergence, since alias errors occur at period estimation before the criticker.

2. Error origin shapes propagation (RQ i, O1): Planner-introduced errors propagate silently to the final output more often than engineer-introduced errors, since planner outputs are high-level and harder to verify numerically.

3. Trajectory metrics should degrade before output metrics (RQ ii, O3): anchor violation rate and retry-success rate should worsen one stress level before final pass rates drop. This gap should be larger in astrophysics, where wrong intermediate values can still yield statistically plausible fits, and smaller in Lean, where the kernel checks each proof step.

4. Verification strictness redistributes failures (RQ iii, O2). Lean trajectories are dominated by localisable proof divergences and reasoning loops; astrophysical trajectories exhibit more silent wrong computation and overconfidence, mirroring [6, 5]. Structural detectors are *expected* to generalise across regimes, whereas anchor-based localisation should remain domain-specific.

6. Evaluation Metrics

Metrics are grouped by the objective they support: correctness at anchor sites (O1), detector validation (O2), and trajectory degradation (O3). On astrophysics, STARGAZER’s published single-agent results across frontier backbones [5] are the primary baseline; on Lean, published Lean-agent pass rates of LEANAGENT [13] on FATE [10] and LEANCAT [11] serve the same role where backbone and sampling budget can be matched.

P1. Correctness at anchor sites. **Lean:** *pass@k* ($k \in \{1, 5\}$, kernel-verified), *subgoal-discharge fraction*; closing proofs gated on MATHLIB reuse and absence of `sorry` or `admit`. **Astrophysics:** STARGAZER’s conjunction gate (Δ BIC, RMS, Hungarian-matched parameter score, planet count), with the gap between statistical and physical pass rates as the “good statistics $\neq$ good physics” signal. Each metric is computed at the final submission and at intermediate anchor sites where applicable.

P2. Localisation and detector validation (O1, O2). Ground truth is automatic on both testbeds: the Lean kernel labels every tactic and subgoal; STARGAZER’s forward model [5] recomputes ground-truth values for every intermediate quantity (periodogram peaks, residual RMS, per-component fits, BIC scores). *Localisation accuracy* (O1), computed per trial: the detector’s flagged event matches the anchor-derived first-violation event with precision and recall ≥ 0.8 . *Detector validation* (O2): detectors fire on STARGAZER’s three patterns (alias convergence, perseveration, format fragility) with precision and recall ≥ 0.8 against forward-model-derived labels.

P3. Trajectory and calibration metrics. *Anchor violation rate* and *error-localisation depth* test P3; *retry-success rate* operationalises perseveration in P1; *origin-attribution rate* (fraction of silent-failure trials whose first violated anchor lies on each agent’s subgraph) tests P2; *overconfidence score* (gap between agent-reported confidence and anchor-derived correctness) operationalises silent acceptance in P1. Structural graph metrics (longest path, cycle count, agent betweenness, dead-end fraction) are *exploratory*: their correlation with output failure is tested as part of P4. Every trial is labelled with one or more O2 taxonomy modes; per-configuration mode distributions are reported.

P4. Statistical protocol. All configurations run for $N \geq 10$ trials per task. Differences are tested with paired bootstrap (10 000 resamples) with Bonferroni correction across each metric family; configuration-level claims require a minimum effect size of Cohen’s $d = 0.4$.

7. Challenges and Limitations

1. Compute priority order. Under compute pressure, axes run orthogonally with this priority: (i) single-agent REACT vs. multi-agent on STARGAZER easy/medium (primary RQ (i) finding); (ii) critic ablation (P1); (iii) stress conditions (O3 early-warning); (iv) backbone ablation if budget remains. Lean ablations piggyback within their testbed.

2. Domain adaptability: The main engineering risk is implementing a compatible trace schema and event-logging infrastructure while keeping domain-specific detectors and metrics well separated. Extension to sparser-anchor regimes such as DISCOVERYBENCH [16] is left to future work.

3. Scope of attribution. The framework localises failures to events and agents but does not explain the model-level cause.

4. Dashboard utility. Visualising long trajectories does not by itself guarantee interpretability; a user study is future work.

Bibliography

- [1] Francisco Villaescusa-Navarro, Boris Bolliet, Pablo Villanueva-Domingo, Adrian E. Bayer, Aidan Acquah, Chetana Amancharla, Almog Barzilay-Siegal, Pablo Bermejo, Camille Bilodeau, Pablo Cárdenas Ramírez, Miles Cranmer, Urbano L. França, ChangHoon Hahn, Yan-Fei Jiang, Raul Jimenez, Jun-Young Lee, Antonio Lerario, Osman Mamun, Thomas Meier, Anupam A. Ojha, Pavlos Protopapas, Shimanto Roy, David N. Spergel, Pedro Tarancón-Álvarez, Ujjwal Tiwari, Matteo Viel, Digvijay Wadekar, Chi Wang, Bonny Y. Wang, Licong Xu, Yossi Yovel, Shuwen Yue, Wen-Han Zhou, Qiyao Zhu, Jiajun Zou, and Íñigo Zubeldia. The denario project: Deep knowledge ai agents for scientific discovery. 2025.
- [2] Licong Xu, Milind Sarkar, Anto I. Lonappan, Íñigo Zubeldia, Pablo Villanueva-Domingo, Santiago Casas, Christian Fidler, Chetana Amancharla, Ujjwal Tiwari, Adrian Bayer, Chadi Ait Ekioui, Miles Cranmer, Adrian Dimitrov, James Fergusson, Kahaan Gandhi, Sven Krippendorf, Andrew Laverick, Julien Lesgourgues, Antony Lewis, Thomas Meier, Blake Sherwin, Kristen Surrao, Francisco Villaescusa-Navarro, Chi Wang, Xueqing Xu, and Boris Bolliet. Open source planning & control system with language agents for autonomous scientific discovery. 2025.
- [3] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using llm agents as research assistants, 2025.
- [4] George Tsoukalas, Anton Kovsharov, Sergey Shirobokov, Anja Surina, Moritz Firsching, Gergely Bérczi, Francisco J. R. Ruiz, Arun Suggala, Adam Zsolt Wagner, Eric Wieser, Lei Yu, Aja Huang, Miklós Z. Horváth, Andrew Ferraiuolo, Henryk Michalewski, Codrut Grosu, Thomas Hubert, Matej Balog, Pushmeet Kohli, and Swarat Chaudhuri. Advancing mathematics research with ai-driven formal proof search, 2026.
- [5] Xinge Liu, Terry Jingchen Zhang, Bernhard Schölkopf, Zhijing Jin, and Kristen Menou. Stargazer: A scalable model-fitting benchmark environment for ai agents under astrophysical constraints, 2026.
- [6] Shivam Rawat and Lucie Flek. Plausible but wrong: A case study on agentic failures in astrophysical workflows, 2025.
- [7] Xixun Lin, Yucheng Ning, Jingwen Zhang, Yan Dong, Yilong Liu, Yongxuan Wu, Xiaohua Qi, Nan Sun, Yanmin Shang, Kun Wang, Pengfei Cao, Qingyue Wang, Lixin Zou, Xu Chen, Chuan Zhou, Jia Wu, Peng Zhang, Qingsong Wen, Shirui Pan, Bin Wang, Yanan Cao, Kai Chen, Songlin Hu, and Li Guo. Llm-based agents suffer from hallucinations: A survey of taxonomy, methods, and directions, 2025.

- [8] Pengfei He, Zhenwei Dai, Bing He, Hui Liu, Xianfeng Tang, Hanqing Lu, Juanhui Li, Jiayuan Ding, Subhabrata Mukherjee, Suhan Wang, Yue Xing, Jiliang Tang, and Benoit Dumoulin. Traject-bench: a trajectory-aware benchmark for evaluating agentic tool use, 2025.
- [9] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635. Springer, 2021.
- [10] Jiedong Jiang, Wanyi He, Yuefeng Wang, Guoxiong Gao, Yongle Hu, Jingting Wang, Nailin Guan, Peihao Wu, Chunbo Dai, Liang Xiao, and Bin Dong. Fate: A formal benchmark series for frontier algebra of multiple difficulty levels, 2025.
- [11] Rongge Xu, Hui Dai, Yiming Fu, Jiedong Jiang, Tianjiao Nie, Junkai Wang, Holiverse Yang, and Zhi-Hao Zhang. Leancat: A benchmark suite for formal category theory in lean (part i: 1-categories), 2025.
- [12] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023.
- [13] Borja Requena, Austin Letson, Krystian Nowakowski, Izan Beltran-Ferreiro, and Leopoldo Sarra. A minimal agent for automated theorem proving, 2026.
- [14] Langfuse: Open source llm engineering platform, 2026. GitHub repository. Accessed: 2026-05-26.
- [15] Understand anything: Interactive knowledge graphs for codebases and knowledge bases, 2026. GitHub repository. Accessed: 2026-05-26.
- [16] Bodhisattwa Prasad Majumder, Harshit Surana, Dhruv Agarwal, Bhavana Dalvi Mishra, Abhijeetsingh Meena, Aryan Prakhar, Tirth Vora, Tushar Khot, Ashish Sabharwal, and Peter Clark. Discoverybench: Towards data-driven discovery with large language models, 2024.